

Database development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 8h50 pm

- Monday A -G209
- Tuesday B-G209
- Wednesday E-G209
- Thursday F-G308

November 2024

Section A: True or False Questions

1. **True or False:** Database cloning refers to duplicating an entire Oracle database, which can be useful for testing without affecting the original database.
2. **True or False:** The SYSDATE function in Oracle returns the current system date and time.
3. **True or False:** The main purpose of a function in PL/SQL is to perform an action, while a procedure is primarily designed to return a result.
4. **True or False:** Cloning a repository with git clone gives you full access to make changes and push them to the remote repository.
5. **True or False:** Cloning in Oracle involves creating an exact copy of a database or its components for tasks such as testing or backup.
6. **True or False:** In the usp_ProcessPayment procedure, nested BEGIN...EXCEPTION...END blocks are used to handle different types of errors, such as insufficient funds and invalid accounts.
7. **True or False:** Non-relational databases store data in structured tables with fixed schemas, just like relational databases
8. **True or False:** the COMMIT statement saves changes made by all SQL statements within the current transaction.
9. **True or False:** The usp_PlaceOrder procedure should raise a custom exception if the order quantity exceeds 100 and handle other errors using the OTHERS exception handler.

Section B: Multiple Choice Questions (MCQs)

1. Why is Oracle Enterprise Manager essential for Oracle database environments?

- a) It improves visual design capabilities
- b) It enhances the security of third-party applications
- c) It ensures the health and efficiency of the Oracle database environment
- d) It is primarily used for software engineering projects

2. What is the benefit of selecting the appropriate datatype for a column?

- a) It increases disk space usage.
- b) It reduces the need for indexing.
- c) It optimizes disk space usage and improves performance.
- d) It allows any data to be inserted regardless of the type.

3. In the usp_CalculateBonus procedure, what happens if an employee's salary is null?

- a) The bonus is calculated based on a default salary value.
- b) The procedure raises an error and handles it.
- c) The procedure continues without calculating the bonus.
- d) The null value is converted to zero, and the bonus is calculated.

4. In which of the following scenarios would you choose PL/SQL over SQL?

- a) To perform simple data retrieval operations
- b) To implement business logic with complex control structures (e.g., loops, conditionals)
- c) Defining database constraints
- d) To grant and revoke permissions on database objects

Section B: Multiple Choice Questions (MCQs)

4. In SQL*Plus, which command type allows you to manipulate and retrieve data from the database?

- a) PL/SQL blocks
- b) Operating System Commands
- c) **SQL Commands**
- d) SQL*Plus Commands

5. Which of the following is NOT a feature of GitHub?

- a) Creating repositories to store and manage code.
- b) Tracking changes in files and managing versions through Git integration.
- c) Hosting static websites using GitHub Pages.
- d) **Automatically writing code and fixing bugs in projects.**

6. Which tablespace is automatically created when the database is created and contains data dictionary tables?

- a) TEMP Tablespace
- b) USERS Tablespace
- c) **SYSTEM Tablespace**
- d) UNDO Tablespace

7. What is the primary difference between SQL and PL/SQL?

- a) SQL is a procedural language, while PL/SQL is a non-procedural language.
- b) **SQL is a non-procedural query language, while PL/SQL is a procedural extension of SQL.**
- c) SQL is used only for managing user permissions, while PL/SQL is used for data manipulation.
- d) SQL cannot be used for querying databases, while PL/SQL is used for querying databases.

Section B: Multiple Choice Questions (MCQs)

8. Which of the following statements is TRUE?

- a) A composite key is always a super key that consists of only one column.
- b) A super key is a set of columns that uniquely identify a row, and it can contain extra attributes not needed for unique identification.
- c) An alternate key is any key that is not unique and cannot serve as a backup identifier.
- d) A candidate key can include additional attributes beyond what is necessary to uniquely identify a row.

9. Which of the following is TRUE about Oracle's Multitenant Architecture?

- a) In Oracle's multitenant architecture, each pluggable database (PDB) operates independently, without sharing resources with the container database (CDB).
- b) The multitenant architecture was introduced in Oracle 11g and supports only a single pluggable database (PDB) within each container database (CDB).
- c) The multitenant architecture allows a container database (CDB) to host multiple pluggable databases (PDBs), enabling easier consolidation and efficient resource management.
- d) Pluggable databases (PDBs) in Oracle's multitenant architecture cannot be upgraded or patched independently of the container database (CDB).

Section C: Required Question – Practical Q-I (1/3)

I. Question

Problem Statement: Calculate VAT for a Business Using a PL/SQL Stored Procedure

The Rwanda Revenue Authority (RRA) requires businesses to calculate and remit Value Added Tax (VAT) based on their total sales. The VAT rate in Rwanda is set at 18%. As a system developer, you are tasked with writing a PL/SQL stored procedure that calculates the VAT for a single business.

You are provided with the following details of the business:

Name	Location	Business Type	Total Sales (RWF)
Trade Business Center LTD	Kigali	Retail	50,000,000

Your Task:

1. Create a PL/SQL stored procedure that calculates the VAT for this business using the provided information.
2. The procedure should:
 - Take Total Sales as input.
 - Calculate VAT (18% of Total Sales).
 - Compute the Total Sales Including VAT.
 - Display the results (business name, location, business type, VAT amount, and total including VAT).
2. Allow the user to modify or update the total sales value as an input parameter to the stored procedure so that they can check how much VAT they would pay if the total sales change.

VAT Calculation:

- VAT Rate: 18% of Total Sales
- VAT Amount = Total Sales * 0.18
- Total Sales Including VAT = Total Sales + VAT Amount

Section C: Required Question – Practical Q-I

(2/3)

Example:

For a business with a total sale of 5,000,000 RWF:

- VAT Amount = $5,000,000 * 0.18 = 900,000$ RWF
- Total Sales Including VAT = $5,000,000 + 900,000 = 5,900,000$ RWF

Requirements:

- The procedure should accept a custom Total Sales value when called, enabling the user to check the amount they would need to pay in VAT for any given sales figure.

Expected Output:

When the stored procedure is called with a Total Sales of 5,000,000 RWF, the output should be as follows:

- Name: Trade Business Center LTD
- Location: Kigali
- Business Type: Retail
- Total Sales (RWF): 5000000
- VAT Amount (RWF): 900000
- Total Sales Including VAT (RWF): 5900000
- **Error Handling:**
 - The stored procedure should handle exceptions that might occur, such as invalid input for sales figures (e.g., negative numbers or non-numeric input).
- **Procedure Call:**
 - Provide an example of how to call the procedure, demonstrating its functionality with a specified Total Sales value.

Solution – Create the procedure

Explanation:

DBMS_OUTPUT.PUT_LINE: This is used in Oracle PL/SQL to print or display output to the console.

'Total Sales (RWF)': This is the text portion of the output, indicating that the following number represents the total sales in Rwandan Francs (RWF).

TO_CHAR(p_total_sales, 'FM999,999,999'):

- TO_CHAR converts the numeric value of p_total_sales into a string, formatted according to the provided pattern.
- The format model 'FM999,999,999':
 - **999,999,999:** Specifies that the number should be formatted with commas as thousands separators, up to 9 digits.
 - **FM (Fill Mode):** Suppresses any leading spaces in the output, making the output cleaner by avoiding extra padding.

```
CREATE OR REPLACE PROCEDURE Calculate_VAT(  
    p_total_sales      IN   NUMBER,      -- Input parameter for Total Sales  
    p_vat_amount       OUT  NUMBER,      -- Output parameter for VAT amount  
    p_total_sales_incl_vat OUT  NUMBER    -- Output parameter for Total Sales Including VAT  
)  
IS  
    -- Constants for business details and VAT rate  
    v_business_name CONSTANT VARCHAR2(50) := 'Trade Business Center LTD';  
    v_location CONSTANT VARCHAR2(50) := 'Kigali';  
    v_business_type CONSTANT VARCHAR2(50) := 'Retail';  
    v_vat_rate CONSTANT NUMBER := 0.18;  
  
BEGIN  
    -- Validate input  
    IF p_total_sales <= 0 THEN  
        RAISE_APPLICATION_ERROR(-20001, 'Total Sales must be a positive number.');    END IF;  
  
    -- Calculate VAT amount and Total Sales Including VAT  
    p_vat_amount := p_total_sales * v_vat_rate;  
    p_total_sales_incl_vat := p_total_sales + p_vat_amount;  
  
    -- Display results  
    DBMS_OUTPUT.PUT_LINE('Business Name: ' || v_business_name);  
    DBMS_OUTPUT.PUT_LINE('Location: ' || v_location);  
    DBMS_OUTPUT.PUT_LINE('Business Type: ' || v_business_type);  
    DBMS_OUTPUT.PUT_LINE('Total Sales (RWF): ' || TO_CHAR(p_total_sales, '999,999,999'));  
    DBMS_OUTPUT.PUT_LINE('VAT Amount (RWF): ' || TO_CHAR(p_vat_amount, '999,999,999'));  
    DBMS_OUTPUT.PUT_LINE('Total Sales Including VAT (RWF): ' || TO_CHAR(p_total_sales_incl_vat,  
'999,999,999'));  
EXCEPTION  
    WHEN VALUE_ERROR THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Invalid input. Please enter a numeric value for Total Sales.');    WHEN OTHERS THEN  
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);  
END Calculate_VAT;  
/
```


Solution – Call the procedure

DBMS_OUTPUT.PUT_LINE: This is used in Oracle PL/SQL to print or display output to the console.

'Total Sales (RWF): ': This is the text portion of the output, indicating that the following number represents the total sales in Rwandan Francs (RWF).

TO_CHAR(p_total_sales, 'FM999,999,999'):

- TO_CHAR converts the numeric value of p_total_sales into a string, formatted according to the provided pattern.
- The format model 'FM999,999,999':
 - **999,999,999**: Specifies that the number should be formatted with commas as thousands separators, up to 9 digits.
 - **FM (Fill Mode)**: Suppresses any leading spaces in the output, making the output cleaner by avoiding extra padding.

```
DECLARE
    v_total_sales NUMBER := 5000000;      -- Input total sales
    v_vat_amount NUMBER;                  -- Variable to store VAT amount
    v_total_sales_incl_vat NUMBER;        -- Variable to store Total Sales
Including VAT
BEGIN
    -- Call the procedure
    Calculate_VAT(v_total_sales, v_vat_amount, v_total_sales_incl_vat);

    -- Display the results
    DBMS_OUTPUT.PUT_LINE('Calculated VAT Amount: ' ||
TO_CHAR(v_vat_amount, '999,999,999'));
    DBMS_OUTPUT.PUT_LINE('Total Sales Including VAT: ' ||
TO_CHAR(v_total_sales_incl_vat, '999,999,999'));
END;
/
```

Expected output

Business Name: Trade Business Center LTD
Location: Kigali
Business Type: Retail
Total Sales (RWF): 5,000,000
VAT Amount (RWF): 900,000
Total Sales Including VAT (RWF): 5,900,000
Calculated VAT Amount: 900,000
Total Sales Including VAT: 5,900,000

Section C: Required Question – Practical – Function Q-II

I. Question

Problem Statement

At the Adventist University of Central Africa (AUCA), students who pay their school fees on time, at least **one week before the due date**, are eligible for a **0.2% reduction** in their total school fees. The university wants to implement this deduction as a function in **PL/SQL (Oracle)**, which will calculate the reduced fee based on the payment date and the total school fees.

The function should take the following input:

- `payment_date`: The date on which the payment is made.
- `due_date`: The last date by which payment is expected.
- `total_fees`: The total school fees for the student.

The function should:

1. **Return the total fees with a 0.2% deduction** if the payment is made at least 7 days (one week) before the due date.
2. **Return the full total fees** without any deduction if the payment is made within the last week or later.
3. **Handle exceptions** for cases such as invalid date formats, negative fee amounts, or null values.

Function Requirements:

- **Inputs:** `payment_date` (DATE), `due_date` (DATE), `total_fees` (NUMBER)
- **Outputs:** The reduced school fee or the original fee if no deduction is applicable.
- **Exception Handling:** Handle errors for invalid inputs, such as:
 - Invalid date formats
 - Negative or null `total_fees`
 - Null `payment_date` or `due_date`

Solution – Create a function

```
CREATE OR REPLACE FUNCTION Calculate_Reduced_Fees(
    payment_date IN DATE,
    due_date IN DATE,
    total_fees IN NUMBER
) RETURN NUMBER
IS
    discounted_fee NUMBER;
BEGIN
    -- Validate inputs
    IF payment_date IS NULL OR due_date IS NULL THEN
        RAISE_APPLICATION_ERROR(-20001, 'Payment date and due date cannot be null.');
```

);

```
    ELSIF total_fees IS NULL OR total_fees < 0 THEN
        RAISE_APPLICATION_ERROR(-20002, 'Total fees must be a positive number.');
```

END IF;

```
    -- Calculate discount eligibility
    IF due_date - payment_date >= 7 THEN
        discounted_fee := total_fees - (total_fees * 0.002); -- Apply 0.2% discount
    ELSE
        discounted_fee := total_fees; -- No discount
    END IF;
```

```
    RETURN discounted_fee;
```

```
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An unexpected error occurred: ' || SQLERRM);
        RETURN NULL; -- Return NULL in case of an error
END Calculate_Reduced_Fees;
/
```

Solution – Call the function

```
DECLARE
    -- Assign a sample payment date for testing (format: YYYY-MM-DD)
    payment_date DATE := TO_DATE('2024-02-10', 'YYYY-MM-DD');

    -- Assign the due date for the payment (format: YYYY-MM-DD)
    due_date DATE := TO_DATE('2024-02-17', 'YYYY-MM-DD');

    -- Set the total fees amount for the student
    total_fees NUMBER := 1000000;

    -- Variable to store the calculated reduced fee after applying any eligible
    discount
    reduced_fee NUMBER;

BEGIN
    -- Call the Calculate_Reduced_Fees function to get the reduced fee if
    eligible
    reduced_fee := Calculate_Reduced_Fees(payment_date, due_date,
    total_fees);

    -- Output the result, formatted for readability
    DBMS_OUTPUT.PUT_LINE('The reduced fee is: ' || TO_CHAR(reduced_fee,
    '999,999.99'));
END;
/
```

Section C: Required Question – Practical Q-III

III. Question:

You are tasked with creating a stored procedure in Oracle that calculates and outputs a student's grade based on their score, while also handling specific error cases.

Instructions:

1. Create a stored procedure named `calculate_grade_user_defined_excp` that:
 - Accepts a student's score as an input parameter (a number).
 - Uses the following grading scale:

MARKS %	DESCRIPTION	GRADE
90-100	Excellent	A
80-89	Very Good	B
70-79	Good	C
60-69	Satisfactory	D
BELOW 60	Fail	F

- Outputs the calculated grade using `DBMS_OUTPUT.PUT_LINE`.
 - Raises and handles the following user-defined exceptions:
 - `null_score`: When the input score is null.
 - `invalid_score`: When the input score is less than 0 or greater than 100.
2. Test your procedure with the following cases and capture the output:
 - A valid score of 85.
 - An invalid score of -5.
 - A NULL value for the score.
 - A score of 101.

Ensure your exception handling properly raises the errors and displays the appropriate error messages.

Solution – Calculate Grade

```
CREATE OR REPLACE PROCEDURE calculate_grade_user_defined_excpt (p_score NUMBER) IS
    -- Define user-defined exceptions
    null_score EXCEPTION;
    invalid_score EXCEPTION;

    -- Assign exception numbers to user-defined exceptions
    PRAGMA EXCEPTION_INIT(null_score, -20001);
    PRAGMA EXCEPTION_INIT(invalid_score, -20002);

    v_grade VARCHAR2(20);

BEGIN
    -- Check if the score is NULL and raise null_score exception if true
    IF p_score IS NULL THEN
        RAISE null_score;
    END IF;

    -- Check if the score is outside the valid range (0 to 100) and raise invalid_score exception
    IF p_score < 0 OR p_score > 100 THEN
        RAISE invalid_score;
    END IF;

    -- Determine grade based on the score range
    IF p_score BETWEEN 90 AND 100 THEN
        v_grade := 'A - Excellent';
    ELSIF p_score BETWEEN 80 AND 89 THEN
        v_grade := 'B - Very Good';
    ELSIF p_score BETWEEN 70 AND 79 THEN
        v_grade := 'C - Good';
    ELSIF p_score BETWEEN 60 AND 69 THEN
        v_grade := 'D - Satisfactory';
    ELSE
        v_grade := 'F - Fail';
    END IF;

    -- Output the calculated grade
    DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);

    -- Exception handling block
EXCEPTION
    WHEN null_score THEN
        DBMS_OUTPUT.PUT_LINE('Error: Score cannot be NULL.');
```

```
    WHEN invalid_score THEN
        DBMS_OUTPUT.PUT_LINE('Error: Score must be between 0 and 100.');
```

```
END calculate_grade_user_defined_excpt;
```

Solution – Calculate Grade II Using case

```
CREATE OR REPLACE PROCEDURE calculate_grade_user_defined_excp (p_score NUMBER) IS
    -- Define user-defined exceptions
    null_score EXCEPTION;
    invalid_score EXCEPTION;

    -- Assign exception numbers to user-defined exceptions
    PRAGMA EXCEPTION_INIT(null_score, -20001);
    PRAGMA EXCEPTION_INIT(invalid_score, -20002);

    v_grade VARCHAR2(20);

BEGIN
    -- Check if the score is NULL and raise null_score exception if true
    IF p_score IS NULL THEN
        RAISE null_score;
    END IF;

    -- Check if the score is outside the valid range (0 to 100) and raise invalid_score exception
    IF p_score < 0 OR p_score > 100 THEN
        RAISE invalid_score;
    END IF;

    -- Determine grade based on the score range using CASE
    v_grade := CASE
        WHEN p_score BETWEEN 90 AND 100 THEN 'A - Excellent'
        WHEN p_score BETWEEN 80 AND 89 THEN 'B - Very Good'
        WHEN p_score BETWEEN 70 AND 79 THEN 'C - Good'
        WHEN p_score BETWEEN 60 AND 69 THEN 'D - Satisfactory'
        ELSE 'F - Fail'
    END;

    -- Output the calculated grade
    DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);

    -- Exception handling block
EXCEPTION
    WHEN null_score THEN
        DBMS_OUTPUT.PUT_LINE('Error: Score cannot be NULL.');
```

When invalid_score THEN
DBMS_OUTPUT.PUT_LINE('Error: Score must be between 0 and 100.');

END calculate_grade_user_defined_excp;
/

Solution – Calculate Grade III Using case

```
CREATE OR REPLACE PROCEDURE calculate_grade_user_defined_excp (p_score NUMBER) IS
    -- Define user-defined exceptions
    null_score EXCEPTION;
    invalid_score EXCEPTION;

    -- Assign exception numbers to user-defined exceptions
    PRAGMA EXCEPTION_INIT(null_score, -20001);
    PRAGMA EXCEPTION_INIT(invalid_score, -20002);

    v_grade VARCHAR2(20);

BEGIN
    -- Check if the score is NULL and raise null_score exception if true
    IF p_score IS NULL THEN
        RAISE null_score;
    END IF;

    -- Check if the score is outside the valid range (0 to 100) and raise invalid_score exception
    IF p_score < 0 OR p_score > 100 THEN
        RAISE invalid_score;
    END IF;

    -- Determine grade based on the score range using CASE
    CASE
        WHEN p_score BETWEEN 90 AND 100 THEN v_grade := 'A - Excellent';
        WHEN p_score BETWEEN 80 AND 89 THEN v_grade := 'B - Very Good';
        WHEN p_score BETWEEN 70 AND 79 THEN v_grade := 'C - Good';
        WHEN p_score BETWEEN 60 AND 69 THEN v_grade := 'D - Satisfactory';
        ELSE v_grade := 'F - Fail';
    END CASE;

    -- Output the calculated grade
    DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);

    -- Exception handling block
    EXCEPTION
        WHEN null_score THEN
            DBMS_OUTPUT.PUT_LINE('Error: Score cannot be NULL.');
        WHEN invalid_score THEN
            DBMS_OUTPUT.PUT_LINE('Error: Score must be between 0 and 100.');
    END calculate_grade_user_defined_excp;
/
```


Call the Procedure

– Calculate Grade

```
-- Test with a valid score of 85
BEGIN
    calculate_grade_user_defined_excp(85);
END;
/

-- Test with an invalid score of -5
BEGIN
    calculate_grade_user_defined_excp(-5);
END;
/

-- Test with a NULL score
BEGIN
    calculate_grade_user_defined_excp(NULL);
END;
/

-- Test with a score of 101
BEGIN
    calculate_grade_user_defined_excp(101);
END;
/
```

Section C: Required Question – Practical Q-IV

IV. Question

Procedure to Calculate Area of a Rectangle

You are tasked with creating a **stored procedure** in Oracle that calculates and outputs the area of a rectangle based on its length and width, while handling specific error cases.

Instructions:

1. **Create a stored procedure** named `calculate_area_rectangle` that:
 - Accepts two input parameters:
 - `p_length` NUMBER: The length of the rectangle.
 - `p_width` NUMBER: The width of the rectangle.
 - Calculates the area of the rectangle using the formula:
Area = length × width
 - Outputs the calculated area using `DBMS_OUTPUT.PUT_LINE`.
 - **Raises and handles** the following exceptions:
 - `negative_value`: When either the length or width is negative.
 - `null_value`: When either the length or width is NULL.
2. **Test your procedure** with the following cases and capture the output:
 - A valid length of 10 and width of 5.
 - A negative length (-3) and valid width (5).
 - A NULL value for the width.
 - Both length and width as 0.
3. Ensure that your procedure raises the appropriate exceptions and displays the correct error messages.

Solution – Rectangle Area

```
CREATE OR REPLACE PROCEDURE calculate_area_rectangle (  
    p_length NUMBER,  
    p_width NUMBER  
) IS  
    -- Define user-defined exceptions  
    negative_value EXCEPTION;  
    null_value EXCEPTION;  
  
    -- Variable to store the area  
    v_area NUMBER;  
  
BEGIN  
    -- Check if length or width is NULL and raise null_value exception  
    IF p_length IS NULL OR p_width IS NULL THEN  
        RAISE null_value;  
    END IF;  
  
    -- Check if length or width is negative and raise negative_value exception  
    IF p_length < 0 OR p_width < 0 THEN  
        RAISE negative_value;  
    END IF;  
  
    -- Calculate the area  
    v_area := p_length * p_width;  
  
    -- Output the calculated area  
    DBMS_OUTPUT.PUT_LINE('Area of the rectangle: ' || v_area);  
  
    -- Exception handling block  
EXCEPTION  
    WHEN null_value THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Length and width cannot be NULL.');    WHEN negative_value THEN  
        DBMS_OUTPUT.PUT_LINE('Error: Length and width cannot be negative.');END calculate_area_rectangle;  
/
```

Call the procedure – Calculate Rectangle Area

```
-- Test with a valid length of 10 and width of 5
BEGIN
    calculate_area_rectangle(10, 5);
END;
/

-- Test with a negative length (-3) and valid width (5)
BEGIN
    calculate_area_rectangle(-3, 5);
END;
/

-- Test with a NULL value for the width
BEGIN
    calculate_area_rectangle(10, NULL);
END;
/

-- Test with both length and width as 0
BEGIN
    calculate_area_rectangle(0, 0);
END;
/
```

Section C: Required Question – Practical Q-V

V. Question:

Task: Creating a Stored Procedure to Display Student Information with Error Handling

Instructions:

You are required to create a PL/SQL stored procedure named `display_all_student_pr` that fulfills the following requirements:
The procedure should display the following details for all students from the provided table:

ID	Name	Age	Grade
102	Kalisa Claude	25	A
104	Kamanzi Liliane	20	B
106	Ndayisenga Valens	19	C

Your Task:

The procedure should include error handling to manage any potential exceptions that may arise during execution.

Expected Output: When the procedure is executed, it should display the following format for each student:

- Student ID: [ID]
- Student Name: [Name]
- Age: [Age]
- Grade: [Grade]

Solution – Procedure Display All students

```
CREATE OR REPLACE PROCEDURE display_all_student_pr IS
BEGIN
    -- Use a FOR loop to iterate over each student record returned by the SELECT
    statement
    FOR student IN (
        SELECT ID, Name, Age, Grade
        FROM students -- Replace 'students' with the actual table name if different
    ) LOOP
        -- Display each student's details in the specified format
        DBMS_OUTPUT.PUT_LINE('Student ID: ' || student.ID);
        DBMS_OUTPUT.PUT_LINE('Student Name: ' || student.Name);
        DBMS_OUTPUT.PUT_LINE('Age: ' || student.Age);
        DBMS_OUTPUT.PUT_LINE('Grade: ' || student.Grade);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student data found.');
```

When NO_DATA_FOUND is raised, the procedure will output "No student data found."

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    END display_all_student_pr;
/
```

```
-- Call the procedure
BEGIN
    display_all_student_pr;
END;
/
```

Solution – Procedure Display All students

```
CREATE OR REPLACE PROCEDURE display_all_student_pr IS
BEGIN
    -- Use a FOR loop to iterate through each row returned by the SELECT statement
    FOR student_record IN (
        SELECT ID, Name, Age, Grade
        FROM students -- Replace 'students' with the actual table name if different
    ) LOOP
        -- Display student details in the required format
        DBMS_OUTPUT.PUT_LINE('Student ID: ' || student_record.ID);
        DBMS_OUTPUT.PUT_LINE('Student Name: ' || student_record.Name);
        DBMS_OUTPUT.PUT_LINE('Age: ' || student_record.Age);
        DBMS_OUTPUT.PUT_LINE('Grade: ' || student_record.Grade);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student data found.');
```

```
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
    END display_all_student_pr;
/
```

```
-- Call the procedure
BEGIN
    display_all_student_pr;
END;
/
```

Solution – Procedure Display All students

```
CREATE OR REPLACE PROCEDURE display_all_student_pr IS
    -- Define cursor to select student details
    CURSOR student_cursor IS
        SELECT ID, Name, Age, Grade
        FROM students; -- Replace 'students' with your actual table name if different

    -- Declare variables to hold student details
    v_id students.ID%TYPE;
    v_name students.Name%TYPE;
    v_age students.Age%TYPE;
    v_grade students.Grade%TYPE;

BEGIN
    -- Open cursor and fetch student details
    OPEN student_cursor;
    LOOP
        FETCH student_cursor INTO v_id, v_name, v_age, v_grade;
        EXIT WHEN student_cursor%NOTFOUND;

        -- Display student details
        DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_id);
        DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_name);
        DBMS_OUTPUT.PUT_LINE('Age: ' || v_age);
        DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grade);
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

    -- Close cursor after processing
    CLOSE student_cursor;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student data found.');
```

WHEN OTHERS THEN
DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END display_all_student_pr;
/

Solution using Bulk Collect All students

```
CREATE OR REPLACE PROCEDURE display_all_student_pr IS
    TYPE t_id IS TABLE OF students.ID%TYPE;
    TYPE t_name IS TABLE OF students.Name%TYPE;
    TYPE t_age IS TABLE OF students.Age%TYPE;
    TYPE t_grade IS TABLE OF students.Grade%TYPE;

    v_ids t_id;
    v_names t_name;
    v_ages t_age;
    v_grades t_grade;
BEGIN
    -- Bulk collect data into collections
    SELECT ID, Name, Age, Grade
    BULK COLLECT INTO v_ids, v_names, v_ages, v_grades
    FROM students;

    -- Display each student's details using DBMS_OUTPUT.PUT_LINE
    FOR i IN v_ids.FIRST .. v_ids.LAST LOOP
        DBMS_OUTPUT.PUT_LINE('Student ID: ' || v_ids(i));
        DBMS_OUTPUT.PUT_LINE('Student Name: ' || v_names(i));
        DBMS_OUTPUT.PUT_LINE('Age: ' || v_ages(i));
        DBMS_OUTPUT.PUT_LINE('Grade: ' || v_grades(i));
        DBMS_OUTPUT.PUT_LINE('-----');
    END LOOP;

EXCEPTION
    WHEN NO_DATA_FOUND THEN
        DBMS_OUTPUT.PUT_LINE('No student data found.');
```

When Others Then

```
        DBMS_OUTPUT.PUT_LINE('An error occurred: ' || SQLERRM);
END display_all_student_pr;
/
```

Section C: Required Question – Practical Q-VI

I. Question

Task: Create a Function to Check if a student is an Adult

Instructions:

You are required to create a PL/SQL function named `is_student_adult` that performs the following tasks:

1. Function Purpose: The function should check if a student is considered an adult based on their age.

Input Parameter:

- The function should accept a single input parameter of type `NUMBER`, which represents the age of the student.

2. Return Value:

- The function should return a `VARCHAR2` string indicating whether the student is an adult or not.
- The return values should be:
 - 'Yes, the student is an adult.' if the age is 18 years or older.
 - 'No, the student is not an adult.' if the age is less than 18 years.

3. Error Handling:

- Include appropriate error handling to manage any unexpected issues that may arise during execution (e.g., invalid input values).
- **Example of Function Execution:**
 - If the input age is 20, the function should return: **'Yes, the student is an adult.'**

If the input age is 16, the function should return: **'No, the student is not an adult.'**

Solution – Age II As Function

```
CREATE OR REPLACE FUNCTION is_student_adult(p_age NUMBER)
RETURN VARCHAR2
IS
BEGIN
    -- Check if the input age is negative
    IF p_age < 0 THEN
        -- Return an error message if the age is negative
        RETURN 'Error: Age cannot be negative.';
    ELSIF p_age >= 18 THEN
        -- Return a message confirming the student is an adult
        RETURN 'Yes, the student is an adult.';
    ELSE
        -- Return a message indicating the student is not an adult
        RETURN 'No, the student is not an adult.';
    END IF;
EXCEPTION
    -- Handle any unexpected errors during execution
    WHEN OTHERS THEN
        RETURN 'Error: An unexpected error occurred.';
END is_student_adult;
/
```

```
-- Test with age 20
SELECT is_student_adult(20) AS result FROM dual;

-- Test with age 16
SELECT is_student_adult(16) AS result FROM dual;

-- Test with a negative age
SELECT is_student_adult(-5) AS result FROM dual;
```

Solution – Age I As Procedure

```
CREATE OR REPLACE PROCEDURE is_student_adult(p_age NUMBER)
IS
BEGIN
    -- Check if the input age is negative
    IF p_age < 0 THEN
        -- Output an error message if the age is negative
        DBMS_OUTPUT.PUT_LINE('Error: Age cannot be negative.');
```

ELSIF p_age >= 18 THEN

-- Output a message confirming the student is an adult

DBMS_OUTPUT.PUT_LINE('Yes, the student is an adult.');

ELSE

-- Output a message indicating the student is not an adult

DBMS_OUTPUT.PUT_LINE('No, the student is not an adult.');

END IF;

EXCEPTION

-- Handle any unexpected errors during execution

WHEN OTHERS THEN

DBMS_OUTPUT.PUT_LINE('Error: An unexpected error occurred.');

END is_student_adult;

/

```
-- Test with age 20
SELECT is_student_adult(20) AS result FROM dual;
```

-- Test with age 16

SELECT is_student_adult(16) AS result FROM dual;

-- Test with a negative age

SELECT is_student_adult(-5) AS result FROM dual;

Section C: Required Question – Practical Q-VII

VII. Question

You are given the following employee data stored on a table:

Employee_ID	Name	Country	Department	Job_Title	Salary
101	John Rukundo	Rwanda	IT	Developer	1,200,000
102	Alice Mwiza	Uganda	HR	Manager	2,000,000
103	Didier Mugenzi	Rwanda	Sales	Sales Manager	1,500,000

Your Task:

1. Create a PL/SQL stored procedure to retrieve employees based on the provided Table.
2. The procedure should:
 - Take Country as an input parameter.
 - Select and display the employee details (**Employee_ID, Name, Department, Job_Title, and Salary**) of all employees who belong to the specified country.
 - Handle cases where no employees are found in the specified country and display a message indicating that no employees were found.

Requirements:

- The procedure should take a Country as an input parameter.
- If no employees are found for the given country, display an appropriate message.
- The output should display all employee details (**Employee_ID, Name, Department, Job_Title, and Salary**) for the specified country.
- Error Handling:
 - The procedure should handle the following:
 - If no employees exist for the given country, display a message instead of returning an empty result set.

Solution – Retrieve all Employee by country

```
CREATE OR REPLACE PROCEDURE retrieve_employees_by_country(p_country IN VARCHAR2)
IS
    -- Declare variables to hold employee details
    v_employee_id Employees.Employee_ID%TYPE;
    v_name Employees.Name%TYPE;
    v_department Employees.Department%TYPE;
    v_job_title Employees.Job_Title%TYPE;
    v_salary Employees.Salary%TYPE;

    -- Variable to count employees found
    v_count NUMBER;
BEGIN
    -- Count the number of employees from the specified country
    SELECT COUNT(*)
    INTO v_count
    FROM Employees
    WHERE Country = p_country;

    -- Check if any employees were found
    IF v_count = 0 THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the specified country: ' ||
p_country);
    ELSE
        -- Display details of each employee from the specified country
        FOR emp_record IN (SELECT Employee_ID, Name, Department, Job_Title, Salary
        FROM Employees
        WHERE Country = p_country) LOOP
            DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.Employee_ID ||
                ', Name: ' || emp_record.Name ||
                ', Department: ' || emp_record.Department ||
                ', Job Title: ' || emp_record.Job_Title ||
                ', Salary: ' || emp_record.Salary);
        END LOOP;
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Error: An unexpected error occurred: ' || SQLERRM);
END retrieve_employees_by_country;
/
```

Solution – Procedure retrieve all employees by country

```
CREATE OR REPLACE PROCEDURE retrieve_employees_by_country(p_country IN VARCHAR2) IS
-- Define a cursor to hold the results of the query
CURSOR emp_cursor IS
    SELECT Employee_ID, Name, Department, Job_Title, Salary
    FROM Employees
    WHERE Country = p_country;

-- Variable to hold the employee details
emp_record emp_cursor%ROWTYPE;
employees_found BOOLEAN := FALSE; -- Flag to check if any employees are found
BEGIN
    -- Open the cursor and loop through the results
    OPEN emp_cursor;
    LOOP
        FETCH emp_cursor INTO emp_record;
        EXIT WHEN emp_cursor%NOTFOUND;

        -- Set the flag to true if at least one employee is found
        employees_found := TRUE;

        -- Display the employee details
        DBMS_OUTPUT.PUT_LINE('Employee ID: ' || emp_record.Employee_ID ||
                               ', Name: ' || emp_record.Name ||
                               ', Department: ' || emp_record.Department ||
                               ', Job Title: ' || emp_record.Job_Title ||
                               ', Salary: ' || emp_record.Salary);
    END LOOP;

    -- Close the cursor
    CLOSE emp_cursor;

    -- Check if no employees were found and display a message
    IF NOT employees_found THEN
        DBMS_OUTPUT.PUT_LINE('No employees found for the specified country: ' || p_country);
    END IF;

    EXCEPTION
        WHEN OTHERS THEN
            DBMS_OUTPUT.PUT_LINE('Error: An unexpected error occurred: ' || SQLERRM);
    END retrieve_employees_by_country;
/
```

Call the procedure

```
BEGIN
  -- Call the procedure with the specified country
  retrieve_employees_by_country('Rwanda');
END;
/
```


Section C: Required Question – Practical Q-VIII

VII. Question

Create a PL/SQL Function to Handle Division with Error Handling

You are required to write a PL/SQL function that divides two numbers and handles exceptions, such as division by zero. The function should accept two input parameters, num1 and num2, which represent the numbers to be divided. It should return the result of the division if no error occurs, or handle and report the exception if an error is raised (e.g., when dividing by zero).

Your Task:

1. Create a PL/SQL function named Safe_Divide that:
 - Accepts two numbers (num1 and num2) as input parameters.
 - Attempts to divide num1 by num2.
 - If the division is successful, return the result of the division.
 - If an error occurs (such as division by zero), handle the exception and return an error message.

Requirements:

- Use exception handling to catch the division by zero error (or any other unexpected errors).
- The function should return either the result of the division or an appropriate error message.
- Include logging of the error code and error message in the output if an error occurs.

Function Outline:

- Input Parameters:
 - num1 (Number): The numerator for division.
 - num2 (Number): The denominator for division.
- Output:
 - The result of num1 divided by num2, or an error message if division by zero occurs.

Solution

```
CREATE OR REPLACE FUNCTION Safe_Divide(num1 NUMBER, num2 NUMBER)
RETURN VARCHAR2 AS
    -- Variable to hold the result of the division
    result VARCHAR2(100);
BEGIN
    -- Attempt to perform the division
    result := TO_CHAR(num1 / num2); -- Convert the result to VARCHAR2 for
consistent return type
    RETURN 'Result: ' || result;

EXCEPTION
    -- Handle division by zero specifically
    WHEN ZERO_DIVIDE THEN
        RETURN 'Error: Division by zero is not allowed.';
    -- Handle any other unexpected errors
    WHEN OTHERS THEN
        RETURN 'Error Code: ' || SQLCODE || ', Error Message: ' || SQLERRM;
END Safe_Divide;
/
```

Call the function

```
BEGIN
```

```
DBMS_OUTPUT.PUT_LINE(Safe_Divide(10, 2)); -- Expected output: Result: 5
```

```
DBMS_OUTPUT.PUT_LINE(Safe_Divide(10, 0)); -- Expected output: Error: Division  
by zero is not allowed.
```

```
DBMS_OUTPUT.PUT_LINE(Safe_Divide(NULL, 2)); -- Expected output with error  
code if NULL is used
```

```
END;
```

```
/
```

Keep Learning!